```
import { useState } from "react";

const agents = [
  {
    id: "security",
    name: "Security Reviewer",
    icon: "🛡️",
    color: "#FF6B35",
    bgGlow: "rgba(255,107,53,0.12)",
    dimension: "Segurança",
    layer: "both",
    scope: [
      "[Back] Secrets/credenciais hardcoded (AWS keys, Supabase keys)",
      "[Back] SQL injection em queries Supabase/PostgreSQL RPC",
      "[Back] Validação de inputs em endpoints Lambda",
      "[Back] Permissões IAM excessivas (least privilege)",
      "[Back] Exposição de PII em logs CloudWatch",
      "[Back] SSRF em integrações externas (Digesto, Vertex AI)",
      "[Front] Auth/JWT token handling e storage",
      "[Front] XSS em renderização de dados judiciais",
      "[Front] CORS policy alignment com Lambda endpoints",
      "[Shared] Sanitização de dados no contrato front↔back",
    ],
    systemPrompt: `You are a Security Reviewer agent for a legal tech monorepo co

CONTEXT: You review code for a platform processing sensitive judicial data (audiê
- /backend: AWS Lambda (Python 3.12), Supabase, Azure Blob, Digesto API, Vertex A
- /frontend: React/TypeScript SPA (revisoes_de_eventos)
- Shared contracts and types between layers

WORKFLOW: You are invoked by the lead engineer (Danillo) who assigns you to revie

YOUR RESPONSIBILITIES:
1. [Backend] Scan for hardcoded secrets, API keys, connection strings
2. [Backend] Identify SQL injection in Supabase RPC calls and raw queries
3. [Backend] Validate IAM policies follow least-privilege
4. [Backend] Flag PII exposure in CloudWatch logs or error responses
5. [Backend] Check SSRF risks in Digesto/Vertex AI integrations
6. [Frontend] Verify JWT storage (no localStorage for tokens)
7. [Frontend] Check XSS vectors in judicial data rendering
8. [Cross-layer] Validate CORS alignment between Lambda URLs and React origin
9. [Cross-layer] Check data sanitization at API boundary

IMPORTANT: Present findings organized by layer (Backend/Frontend/Cross-layer) so
```

```
OUTPUT FORMAT:
- layer: BACKEND | FRONTEND | CROSS_LAYER
- severity: CRITICAL | HIGH | MEDIUM | LOW
- location: file:line
- finding: description
- recommendation: specific fix with code snippet
- cwe_reference: CWE-XXX
- decision_needed: true/false (flags items requiring lead engineer judgment)`,
    triggers: ["Danillo atribui feature/módulo", "Novo endpoint ou componente", "
    outputs: ["Security report por camada", "Cross-layer risk assessment", "Items
  },
  {
    id: "architect",
    name: "Architect Specialist",
    icon: "🏗️",
    color: "#4ECDC4",
    bgGlow: "rgba(78,205,196,0.12)",
    dimension: "Arquitetura",
    layer: "both",
    scope: [
      "[Back] Aderência aos diagramas BPMN 2.0",
      "[Back] Separação: handlers → services → repositories",
      "[Back] Padrões event-driven (EventBridge, SQS)",
      "[Back] Schema Supabase ↔ modelos Python",
      "[Back] Connection management (NullPool + asyncpg)",
      "[Front] Component architecture (separation of concerns)",
      "[Front] State management e data flow",
      "[Front] API client layer e error boundaries",
      "[Shared] Contratos de API front↔back (types, DTOs)",
      "[Shared] Acoplamento integracao_jusbrasil ↔ revisoes_eventos",
    ],
    systemPrompt: `You are an Architecture Specialist for a legal tech monorepo.

CONTEXT: The system is a monorepo with:
- /backend (integracao_jusbrasil): AWS Lambda Python — Vertex AI, Supabase, Azure
- /frontend (revisoes_de_eventos): React/TypeScript SPA
- Architecture documented via BPMN 2.0 diagrams (draw.io XML)

WORKFLOW: The lead engineer (Danillo) assigns you to validate architecture before

YOUR RESPONSIBILITIES:
1. [Backend] Verify implementations match BPMN 2.0 process definitions
2. [Backend] Enforce separation: handlers → services → repositories
3. [Backend] Validate event-driven patterns (EventBridge, SQS) wiring
4. [Backend] Check Supabase schema ↔ Python dataclass consistency
5. [Backend] Assess connection management (NullPool + asyncpg)
```

```
6. [Frontend] Review component architecture and responsibility boundaries
7. [Frontend] Validate state management patterns
8. [Frontend] Check API client layer isolation and error boundaries
9. [Cross-layer] Validate API contracts match on both sides
10. [Cross-layer] Flag coupling between backend modules and frontend assumptions

MONOREPO-SPECIFIC:
- Shared type definitions should be single-source-of-truth
- API contracts must be explicitly defined, not inferred
- Changes in one layer that break the other = BREAKING

OUTPUT FORMAT:
- layer: BACKEND | FRONTEND | CROSS_LAYER
- category: DRIFT | COUPLING | CONTRACT_MISMATCH | RESILIENCE_GAP | BREAKING
- component: affected module/service/component
- finding: description with BPMN reference if applicable
- impact: what breaks if ignored
- recommendation: architectural fix
- decision_needed: true/false`,
    triggers: ["Danillo atribui feature/módulo", "Nova integração ou serviço", "S
    outputs: ["Architecture review por camada", "Contract validation front↔back",
  },
  {
    id: "python",
    name: "Python Reviewer",
    icon: "🐍",
    color: "#A855F7",
    bgGlow: "rgba(168,85,247,0.12)",
    dimension: "Qualidade Backend",
    layer: "backend",
    scope: [
      "Type hints completos (Python 3.12)",
      "Async/await patterns (asyncio + asyncpg)",
      "Error handling granular",
      "Logging estruturado para CloudWatch",
      "Docstrings Google-style",
      "Padrões Pythonic (dataclasses, generators)",
      "Performance: N+1 queries, batch ops",
      "Conformidade PEP 8, black, ruff",
    ],
    systemPrompt: `You are a Python Code Quality Reviewer for the backend layer o

CONTEXT: Reviewing /backend Python code for Lambda functions using:
- asyncpg with NullPool for Supabase/PostgreSQL
- Vertex AI SDK, Azure Blob Storage SDK, Digesto API client

WORKFLOW: The lead engineer assigns you to review specific Python files or module
```

```
YOUR RESPONSIBILITIES:
1. Enforce complete type hints (Python 3.12: list[str], X | None)
2. Validate async/await — no blocking calls in async context
3. Check error handling (no bare except:, proper CancelledError handling)
4. Verify structured logging (JSON for CloudWatch)
5. Review Lambda handler patterns: cold start, cleanup/dispose
6. Flag N+1 queries in Supabase interactions
7. Enforce dataclass usage for DTOs
8. Check resource cleanup (async context managers)
9. Validate backend DTOs match shared API contract types

KNOWN ISSUES:
- asyncio.CancelledError in Lambda dispose (must not swallow)
- NullPool required for Lambda
- Secrets Manager fallback to .env for local dev

OUTPUT FORMAT:
- rule: PYXXX
- severity: ERROR | WARNING | INFO
- location: file:line
- finding: description
- fix: code suggestion
- affects_contract: true/false`,
    triggers: ["Danillo atribui review Python", "Novo handler ou service", "Refac
    outputs: ["Code quality report", "Auto-fix suggestions", "Contract impact fla
  },
  {
    id: "typescript",
    name: "TypeScript Reviewer",
    icon: "🔷",
    color: "#3B82F6",
    bgGlow: "rgba(59,130,246,0.12)",
    dimension: "Qualidade Frontend",
    layer: "frontend",
    scope: [
      "TypeScript strict mode compliance",
      "React hooks correctness (deps, rules of hooks)",
      "Component composition e SRP",
      "Tipagem de API responses (types compartilhados)",
      "Performance: memoization, re-renders",
      "Acessibilidade (ARIA, keyboard nav)",
      "Error boundaries e fallback UI",
      "Conformidade ESLint + Prettier",
    ],
    systemPrompt: `You are a TypeScript/React Code Quality Reviewer for the front
```

```
CONTEXT: Reviewing /frontend React/TypeScript SPA (revisoes_de_eventos) that:
- Consumes Lambda backend API endpoints
- Displays judicial data (audiências, atas, processos, movimentos)
- Uses component patterns like AtasTable.tsx with status indicators

WORKFLOW: The lead engineer assigns you to review specific React components or fe

YOUR RESPONSIBILITIES:
1. Enforce TypeScript strict mode — no 'any', proper generics
2. Validate React hooks: dependency arrays, rules of hooks
3. Check component SRP — flag components doing too much
4. Verify API response types match shared contract definitions
5. Flag performance: missing memo/useMemo, unnecessary re-renders
6. Check accessibility: ARIA labels, keyboard nav, focus management
7. Validate error boundaries wrap critical UI sections
8. Check status constants (STATUS_BUSCA) use allowlist pattern

KNOWN PATTERNS:
- "Ata a Lançar" indicator uses allowlist in AtasTable.tsx
- API client should handle 403 from Digesto gracefully

OUTPUT FORMAT:
- rule: TSXXX
- severity: ERROR | WARNING | INFO
- location: file:line
- finding: description
- fix: code suggestion
- affects_contract: true/false`,
    triggers: ["Danillo atribui review React/TS", "Novo componente ou page", "Ref
    outputs: ["Code quality report", "Component suggestions", "Contract impact fl
  },
  {
    id: "qa",
    name: "QA Reviewer",
    icon: "🧪",
    color: "#FBBF24",
    bgGlow: "rgba(251,191,36,0.12)",
    dimension: "Testes",
    layer: "both",
    scope: [
      "[Back] Cobertura pytest + pytest-asyncio",
      "[Back] Mocks realistas (Digesto, Vertex AI)",
      "[Back] Testes integração com test schema",
      "[Front] Jest + React Testing Library",
      "[Front] Testes componentes com mock API",
      "[Shared] Testes de contrato front↔back",
      "[Shared] E2E fluxos críticos (lançamento atas)",
```

```
      "[Shared] Edge cases: nulos, duplicatas, timeouts",
    ],
    systemPrompt: `You are a QA/Test Reviewer for a legal tech monorepo (frontend

CONTEXT: Testing covers both layers:
- /backend: pytest, pytest-asyncio, mocked external APIs
- /frontend: Jest, React Testing Library, MSW for API mocking
- /shared: Contract tests ensuring type agreement

WORKFLOW: The lead engineer assigns you to evaluate test coverage for features.

YOUR RESPONSIBILITIES:
1. [Backend] Verify pytest coverage (min 80%)
2. [Backend] Check mocks match actual API responses
3. [Backend] Ensure integration tests use isolated test schema
4. [Frontend] Verify component tests with RTL
5. [Frontend] Check MSW handlers match backend response shapes
6. [Cross-layer] Validate contract tests exist
7. [Cross-layer] Flag E2E gaps for critical flows
8. [Cross-layer] Check edge cases: null partes_processo, duplicate processamento_

MONOREPO ADVANTAGE:
- Frontend mocks generated from backend response types
- Contract tests auto-fail when types diverge
- E2E tests run against both layers

KNOWN EDGE CASES:
- Missing partes_processo records
- Duplicate processamento_ata
- Digesto API 403 (IP block)
- asyncio.CancelledError during Lambda cleanup

OUTPUT FORMAT:
- layer: BACKEND | FRONTEND | CROSS_LAYER
- test_gap: UNIT | INTEGRATION | E2E | CONTRACT
- component: what's untested
- risk: what could break
- suggested_test: test skeleton
- decision_needed: true/false`,
    triggers: ["Danillo atribui análise de testes", "Feature sem testes", "Bug em
    outputs: ["Coverage report por camada", "Test gap analysis", "Test skeletons"
  },
  {
    id: "devops",
    name: "DevOps Engineer",
    icon: "⚙️",
    color: "#F97316",
```

```
    bgGlow: "rgba(249,115,22,0.12)",
    dimension: "Infraestrutura",
    layer: "infra",
    scope: [
      "Lambda config (memory, timeout, layers)",
      "EventBridge rules e SQS DLQs",
      "CloudWatch alarms e métricas",
      "Terraform/IaC consistency",
      "CI/CD monorepo (build front+back)",
      "Secrets Manager",
      "Docker Compose dev local (full stack)",
      "Cost optimization",
    ],
    systemPrompt: `You are a DevOps Engineer for a legal tech monorepo.

CONTEXT: Infrastructure supports both layers from a single repo:
- /backend: AWS Lambda (Python 3.12), EventBridge, SQS, Secrets Manager
- /frontend: React SPA (static deploy, CDN)
- Shared: Supabase, Azure Blob Storage
- Local dev: Docker Compose for full stack
- Observability: CloudWatch

WORKFLOW: The lead engineer assigns you to review infra or deployment changes.

YOUR RESPONSIBILITIES:
1. Review Lambda config: memory, timeout, concurrency
2. Validate EventBridge rules
3. Check CloudWatch alarms
4. Verify Secrets Manager usage
5. Review monorepo CI/CD: parallel build/test/deploy for both layers
6. Validate Docker Compose runs full stack locally
7. Check SQS DLQ configuration
8. Monitor costs

MONOREPO CI/CD:
- Build: parallel jobs for /backend and /frontend
- Test: both suites + contract tests as gate
- Deploy: backend first, then frontend (CDN invalidation)
- Rollback: independent per layer

OUTPUT FORMAT:
- category: CONFIG | OBSERVABILITY | COST | CI_CD | LOCAL_DEV
- layer: BACKEND | FRONTEND | SHARED
- resource: AWS resource or config path
- finding: description
- risk: operational impact
- fix: IaC snippet or config change`,
```

```
    triggers: ["Danillo atribui review infra", "Mudança CI/CD ou deploy", "Alerta
    outputs: ["Infra review", "CI/CD validation", "Cost report"],
  },
  {
    id: "docs",
    name: "Documentation Specialist",
    icon: "📝",
    color: "#06B6D4",
    bgGlow: "rgba(6,182,212,0.12)",
    dimension: "Documentação",
    layer: "both",
    scope: [
      "README.md monorepo (setup front+back)",
      "Docstrings Python (Google-style)",
      "JSDoc TypeScript",
      "ADRs para decisões arquiteturais",
      "BPMN 2.0 sincronizado com código",
      "API docs (contratos front↔back)",
      "Changelog unificado monorepo",
      "Runbooks operacionais",
    ],
    systemPrompt: `You are a Documentation Specialist for a legal tech monorepo.

CONTEXT: The project maintains:
- BPMN 2.0 diagrams (draw.io XML)
- Technical .docx documents (blue palette, styled tables)
- Python docstrings (Google-style) and TypeScript JSDoc
- Monorepo README with setup for both layers

WORKFLOW: The lead engineer assigns you to check docs for features or changes.

YOUR RESPONSIBILITIES:
1. [Backend] Verify Google-style docstrings
2. [Frontend] Verify JSDoc with @param/@returns
3. [Shared] Check README covers monorepo setup
4. [Shared] Flag BPMN out of sync with code
5. [Shared] Ensure API contract docs exist
6. [Shared] Validate ADRs for significant decisions
7. [Shared] Review changelog entries
8. [Shared] Check runbooks

OUTPUT FORMAT:
- layer: BACKEND | FRONTEND | SHARED
- gap: MISSING | OUTDATED | INCOMPLETE
- artifact: what document is affected
- context: why it matters
- suggestion: content to add`,
```

```
    triggers: ["Danillo atribui review docs", "Novo módulo ou feature", "Schema m
    outputs: ["Doc gap report por camada", "Docstring/JSDoc suggestions", "BPMN s
  },
  {
    id: "compliance",
    name: "Compliance & Dependencies",
    icon: "📦",
    color: "#8B5CF6",
    bgGlow: "rgba(139,92,246,0.12)",
    dimension: "Compliance + Deps",
    layer: "both",
    scope: [
      "[Back] CVEs em requirements.txt",
      "[Front] CVEs em package.json",
      "[Back] Licenças Python packages",
      "[Front] Licenças npm packages",
      "[Shared] LGPD compliance",
      "[Shared] Audit logging",
      "[Shared] Data retention policies",
      "[Shared] SLA monitoring APIs externas",
    ],
    systemPrompt: `You are a Compliance & Dependencies agent for a legal tech mon

CONTEXT: Two dependency trees:
- /backend: requirements.txt (Python)
- /frontend: package.json (npm)
Regulatory: Brazilian legal system, LGPD required.

WORKFLOW: The lead engineer assigns audits across both layers.

YOUR RESPONSIBILITIES:
1. [Backend] Scan requirements.txt for CVEs
2. [Frontend] Scan package.json for CVEs
3. [Both] Verify license compatibility
4. [Both] Flag deprecated packages
5. [Cross-layer] Validate LGPD compliance
6. [Cross-layer] Check audit logging
7. [Cross-layer] Review data retention
8. [Cross-layer] Monitor SLA compliance

LGPD (BOTH LAYERS):
- Backend: access controls on partes data
- Frontend: no PII in browser storage/console
- Shared: audit trail, right to deletion

OUTPUT FORMAT:
- layer: BACKEND | FRONTEND | CROSS_LAYER
```

```
- category: CVE | LICENSE | LGPD | DEPRECATION | SLA
- severity: CRITICAL | HIGH | MEDIUM | LOW
- finding: description
- remediation: specific action
- decision_needed: true/false`,
    triggers: ["Danillo atribui auditoria", "Atualização de deps", "Novo fluxo da
    outputs: ["Vulnerability report (pip+npm)", "License audit", "LGPD checklist"
  },
  {
    id: "patterns",
    name: "Pattern & Trends Analyst",
    icon: "📊",
    color: "#EC4899",
    bgGlow: "rgba(236,72,153,0.12)",
    dimension: "Padrões + Tendências",
    layer: "meta",
    scope: [
      "Correlações cross-agent e cross-layer",
      "Hotspots: arquivos com mais findings",
      "Tendências de qualidade",
      "Tech debt tracking (TODOs, FIXMEs)",
      "Complexidade ciclomática",
      "Padrões de falha produção",
      "Resumo executivo para decisão do lead",
      "Recomendações priorizadas p/ comitê",
    ],
    systemPrompt: `You are a Pattern & Trends Analyst that synthesizes insights f

CONTEXT: You receive findings from 8 agents reviewing a legal tech monorepo. You

WORKFLOW: After all agents complete, you:
1. Aggregate findings across agents and layers
2. Identify correlations and systemic issues
3. Produce a prioritized decision brief

DECISION BRIEF FORMAT:
- 🔴 MUST_FIX: Critical/blocking — fix before proceeding
- 🟡 SHOULD_FIX: High importance — schedule in current sprint
- 🔵 CONSIDER: Medium — discuss with team/Gustavo
- ⚪ NOTED: Low — track for later

For each item include:
- Which agents flagged it
- Which layers affected
- Concrete recommendation
- Whether it needs lead decision or can auto-resolve
```

```
CROSS-AGENT PATTERNS:
- Security + Architecture same component → systemic risk → 🔴
- QA gaps + production incidents → testing strategy issue → 🟡
- Doc gaps + bug density → knowledge gap → 🔵
- Same anti-pattern front+back → monorepo convention needed → 🟡

OUTPUT FORMAT:
- priority: 🔴 MUST_FIX | 🟡 SHOULD_FIX | 🔵 CONSIDER | ⚪ NOTED
- agents_involved: [list]
- layers_affected: BACKEND | FRONTEND | CROSS_LAYER
- finding: cross-cutting observation
- recommendation: specific action
- decision_type: AUTO_RESOLVE | LEAD_DECISION | TEAM_DISCUSSION`,
    triggers: ["Após agentes completarem", "Danillo solicita consolidação", "Retr
    outputs: ["Decision brief para o lead", "Cross-agent report", "Sprint health
  },
];

const orchestrationFlow = [
  { step: 1, label: "Atribuição", desc: "Danillo define o escopo: feature, módulo
  { step: 2, label: "Dispatch", desc: "Orchestrator designa agentes relevantes ao
  { step: 3, label: "Análise Paralela", desc: "Agentes analisam front + back em p
  { step: 4, label: "Consolidação", desc: "Pattern Analyst agrega findings e mont
  { step: 5, label: "Comitê / Decisão", desc: "Danillo analisa o brief e decide o
  { step: 6, label: "Execução", desc: "Implementação das decisões aprovadas no mo
];

const layerColors = {
  backend: { bg: "#A855F720", border: "#A855F7", text: "#A855F7", label: "Back" }
  frontend: { bg: "#3B82F620", border: "#3B82F6", text: "#3B82F6", label: "Front"
  both: { bg: "#4ECDC420", border: "#4ECDC4", text: "#4ECDC4", label: "Full-Stack
  infra: { bg: "#F9731620", border: "#F97316", text: "#F97316", label: "Infra" },
  meta: { bg: "#EC489920", border: "#EC4899", text: "#EC4899", label: "Meta" },
};

const actorStyles = {
  danillo: { bg: "#22c55e18", border: "#22c55e50", label: "Você" },
  system: { bg: "#6366f118", border: "#6366f150", label: "System" },
  agents: { bg: "#f59e0b18", border: "#f59e0b50", label: "Agents" },
};

export default function AgentSquadV2() {
  const [selectedAgent, setSelectedAgent] = useState(null);
  const [activeTab, setActiveTab] = useState("overview");
  const [layerFilter, setLayerFilter] = useState("all");

  const agent = agents.find((a) => a.id === selectedAgent);
```

```
const filteredAgents = layerFilter === "all"
  ? agents
  : agents.filter((a) => a.layer === layerFilter || a.layer === "both" || a.lay

return (
  <div style={{ minHeight: "100vh", background: "#0a0a0f", color: "#e2e2e8", fo

    {/* Header */}
    <div style={{ marginBottom: 28 }}>
      <div style={{ display: "flex", alignItems: "center", gap: 8, marginBottom
        <div style={{ fontSize: 10, textTransform: "uppercase", letterSpacing:
          Human-in-the-Loop • Multi-Agent Review
        </div>
        <div style={{ fontSize: 9, padding: "2px 8px", background: "#22c55e18",
          MONOREPO
        </div>
      </div>
      <h1 style={{ fontSize: 26, fontWeight: 700, color: "#f0f0f5", margin: 0,
        Agent Squad — Ernesto Borges
      </h1>
      <p style={{ color: "#7a7a8a", fontSize: 12, marginTop: 6, maxWidth: 620 }
        9 agentes especializados analisando front + back em paralelo.
        Você atribui, os agentes analisam, e você decide o que sobe.
      </p>
    </div>

    {/* Tabs */}
    <div style={{ display: "flex", gap: 4, marginBottom: 20, flexWrap: "wrap" }
      {[
        { id: "overview", label: "Agentes" },
        { id: "flow", label: "Fluxo" },
        { id: "prompt", label: "System Prompts" },
      ].map((tab) => (
        <button key={tab.id} onClick={() => setActiveTab(tab.id)} style={{
          padding: "7px 14px", fontSize: 11, border: "1px solid",
          borderColor: activeTab === tab.id ? "#6366f1" : "#1e1e2e",
          background: activeTab === tab.id ? "#6366f120" : "transparent",
          color: activeTab === tab.id ? "#a5b4fc" : "#6b6b7b",
          borderRadius: 6, cursor: "pointer", fontFamily: "inherit",
        }}>
          {tab.label}
        </button>
      ))}
    </div>

    {/* OVERVIEW */}
    {activeTab === "overview" && (
```

```
<div>
  <div style={{ display: "flex", gap: 4, marginBottom: 16, flexWrap: "wra
    {[
      { id: "all", label: "Todos (9)" },
      { id: "backend", label: "Backend" },
      { id: "frontend", label: "Frontend" },
      { id: "infra", label: "Infra" },
    ].map((f) => (
      <button key={f.id} onClick={() => setLayerFilter(f.id)} style={{
        padding: "4px 10px", fontSize: 10,
        border: `1px solid ${layerFilter === f.id ? "#4ECDC450" : "#1a1a2
        background: layerFilter === f.id ? "#4ECDC410" : "transparent",
        color: layerFilter === f.id ? "#4ECDC4" : "#5a5a6a",
        borderRadius: 4, cursor: "pointer", fontFamily: "inherit",
      }}>
        {f.label}
      </button>
    ))}
  </div>

  <div style={{ display: "grid", gridTemplateColumns: "repeat(auto-fill,
    {filteredAgents.map((a) => {
      const lc = layerColors[a.layer];
      return (
        <button key={a.id} onClick={() => setSelectedAgent(selectedAgent
          background: selectedAgent === a.id ? a.bgGlow : "#12121a",
          border: `1px solid ${selectedAgent === a.id ? a.color + "60" :
          borderRadius: 10, padding: "14px", cursor: "pointer", textAlign
          transition: "all 0.2s", fontFamily: "inherit", position: "relat
        }}>
          <div style={{ position: "absolute", top: 0, left: 0, width: 3,
          <div style={{ display: "flex", alignItems: "center", justifyCon
            <div style={{ display: "flex", alignItems: "center", gap: 8 }
              <span style={{ fontSize: 18 }}>{a.icon}</span>
              <span style={{ fontSize: 13, fontWeight: 600, color: a.colo
            </div>
            <span style={{ fontSize: 8, padding: "2px 6px", background: l
              {lc.label}
            </span>
          </div>
          <div style={{ fontSize: 10, color: "#6b6b7b", textTransform: "u
          <div style={{ display: "flex", flexWrap: "wrap", gap: 3 }}>
            {a.triggers.map((t, i) => (
              <span key={i} style={{ fontSize: 9, padding: "2px 6px", bac
            ))}
          </div>
        </button>
```

```
          );
        })}
      </div>

      {agent && (
        <div style={{ background: "#12121a", border: `1px solid ${agent.color
          <div style={{ display: "flex", alignItems: "center", gap: 10, margi
            <span style={{ fontSize: 22 }}>{agent.icon}</span>
            <div>
              <h2 style={{ margin: 0, fontSize: 16, color: agent.color }}>{ag
              <span style={{ fontSize: 10, color: "#6b6b7b" }}>{agent.dimensi
            </div>
          </div>
          <div style={{ marginBottom: 14 }}>
            <div style={{ fontSize: 10, color: "#6366f1", textTransform: "upp
            <div style={{ display: "flex", flexDirection: "column", gap: 3 }}
              {agent.scope.map((s, i) => {
                const tagColor = s.startsWith("[Back]") ? "#A855F7" : s.start
                return (
                  <div key={i} style={{ fontSize: 11, color: "#b0b0c0", paddi
                );
              })}
            </div>
          </div>
          <div style={{ display: "flex", gap: 14, flexWrap: "wrap" }}>
            <div style={{ flex: 1, minWidth: 180 }}>
              <div style={{ fontSize: 10, color: "#6366f1", textTransform: "u
              {agent.triggers.map((t, i) => <div key={i} style={{ fontSize: 1
            </div>
            <div style={{ flex: 1, minWidth: 180 }}>
              <div style={{ fontSize: 10, color: "#6366f1", textTransform: "u
              {agent.outputs.map((o, i) => <div key={i} style={{ fontSize: 11
            </div>
          </div>
        </div>
      )}
    </div>
  )}

  {/* FLOW */}
  {activeTab === "flow" && (
    <div>
      <div style={{ padding: "12px 14px", background: "#22c55e0a", border: "1
        👤 <strong>Você é o gate.</strong> Atribui o escopo, os agentes anali
      </div>

      <div style={{ display: "flex", flexDirection: "column", gap: 2, maxWidt
```

```
  {orchestrationFlow.map((step, i) => {
    const actor = actorStyles[step.actor];
    return (
      <div key={step.step}>
        <div style={{ padding: "14px 16px", background: actor.bg, borde
          <div style={{ display: "flex", alignItems: "center", gap: 12,
            <div style={{ width: 34, height: 34, borderRadius: 8, backg
              {step.icon}
            </div>
            <div style={{ flex: 1 }}>
              <div style={{ display: "flex", alignItems: "center", gap:
                <span style={{ fontSize: 13, fontWeight: 600, color: "#
                <span style={{ fontSize: 8, padding: "1px 5px", backgro
              </div>
              <div style={{ fontSize: 11, color: "#8888a0", marginTop:
            </div>
          </div>
          <div style={{ fontSize: 10, color: "#5a5a6a", marginTop: 6, p
        </div>
        {i < orchestrationFlow.length - 1 && (
          <div style={{ display: "flex", justifyContent: "center", padd
            <div style={{ width: 1, height: 14, background: "#2a2a3a" }
          </div>
        )}
      </div>
    );
  })}
</div>

<div style={{ marginTop: 24 }}>
  <div style={{ fontSize: 10, color: "#6366f1", textTransform: "upperca
    Execução Paralela (Step 3) — Por Camada
  </div>
  {[
    { label: "Backend", ags: agents.filter(a => ["backend", "both"].inc
    { label: "Frontend", ags: agents.filter(a => ["frontend", "both"].i
    { label: "Infra", ags: agents.filter(a => a.layer === "infra"), col
    { label: "Meta", ags: agents.filter(a => a.layer === "meta"), color
  ].map((g) => (
    <div key={g.label} style={{ marginBottom: 8 }}>
      <div style={{ fontSize: 9, color: g.color, textTransform: "upperc
      <div style={{ display: "flex", flexWrap: "wrap", gap: 4 }}>
        {g.ags.map((a) => (
          <div key={a.id} style={{ padding: "5px 10px", background: a.b
            <span>{a.icon}</span><span>{a.name}</span>
          </div>
        ))}
```

```
          </div>
        </div>
      ))}
    </div>

    <div style={{ marginTop: 24, padding: 16, background: "#12121a", border
      <div style={{ fontSize: 10, color: "#22c55e", textTransform: "upperca
        ⚖️ Comitê de Decisão (Step 5) — Seu papel
      </div>
      {[
        { label: "🔴 MUST_FIX", desc: "Corrigir antes de prosseguir", actic
        { label: "🟡 SHOULD_FIX", desc: "Agendar no sprint atual", action:
        { label: "🔵 CONSIDER", desc: "Discutir com Gustavo/time", action:
        { label: "⚪ NOTED", desc: "Rastrear para depois", action: "Você re
      ].map((r) => (
        <div key={r.label} style={{ display: "flex", alignItems: "center",
          <span style={{ fontSize: 12, minWidth: 100, fontWeight: 600, colo
          <span style={{ color: "#9a9ab0", flex: 1 }}>{r.desc}</span>
          <span style={{ color: "#22c55e", fontSize: 10, fontStyle: "italic
        </div>
      ))}
    </div>
  </div>
)}

{/* PROMPTS */}
{activeTab === "prompt" && (
  <div>
    <div style={{ fontSize: 11, color: "#8888a0", marginBottom: 14 }}>Selec
    <div style={{ display: "flex", flexWrap: "wrap", gap: 5, marginBottom:
      {agents.map((a) => (
        <button key={a.id} onClick={() => setSelectedAgent(a.id)} style={{
          padding: "5px 10px", fontSize: 10,
          border: `1px solid ${selectedAgent === a.id ? a.color : "#1e1e2e"
          background: selectedAgent === a.id ? a.bgGlow : "transparent",
          color: selectedAgent === a.id ? a.color : "#6b6b7b",
          borderRadius: 5, cursor: "pointer", fontFamily: "inherit",
          display: "flex", alignItems: "center", gap: 5,
        }}>
          <span>{a.icon}</span>{a.name}
        </button>
      ))}
    </div>

    {agent && (
      <div style={{ background: "#0d0d14", border: `1px solid ${agent.color
        <div style={{ display: "flex", alignItems: "center", gap: 8, margin
```

```
                <span style={{ fontSize: 16 }}>{agent.icon}</span>
                <span style={{ fontSize: 13, fontWeight: 600, color: agent.color
                <span style={{ fontSize: 9, padding: "1px 5px", background: layer
                  {layerColors[agent.layer].label}
                </span>
                <span style={{ fontSize: 9, color: "#4b4b5b", marginLeft: "auto"
              </div>
              <pre style={{ fontSize: 10, lineHeight: 1.55, color: "#c0c0d0", whi
                {agent.systemPrompt}
              </pre>
            </div>
          )}
        </div>
      )}

      <style>{`
        @keyframes fadeIn { from { opacity: 0; transform: translateY(8px); } to {
        * { box-sizing: border-box; }
        ::-webkit-scrollbar { width: 5px; }
        ::-webkit-scrollbar-track { background: #0a0a0f; }
        ::-webkit-scrollbar-thumb { background: #2a2a3a; border-radius: 3px; }
      `}</style>
    </div>
  );
}
```